

Code Export

Source Code PDF Report

Generated 2026-03-18 09:09

Root C:\Users\LENOVO\Documents\Java_run\LearningManagementSystem\LearningManagementSystem
directory

Files 52
found

Total lines 2,240

Table of Contents

1. Parent: README.md (3 lines)
2. Current: .mvn\wrapper\maven-wrapper.properties (4 lines)
3. Current: pom.xml (99 lines)
4. Current: src\main\java\com\example\LearningManagementSystem\code\CodeToPdf.java (347 lines)
5. Current: src\main\java\com\example\LearningManagementSystem\Controller\CategoryController.java (47 lines)
6. Current: src\main\java\com\example\LearningManagementSystem\Controller\CourseController.java (69 lines)
7. Current: src\main\java\com\example\LearningManagementSystem\Controller\CourseMaterialController.java (53 lines)
8. Current: src\main\java\com\example\LearningManagementSystem\Controller\EnrollmentController.java (54 lines)
9. Current: src\main\java\com\example\LearningManagementSystem\Controller\GlobalExceptionHandler.java (122 lines)
10. Current: src\main\java\com\example\LearningManagementSystem\Controller\InstructorController.java (47 lines)
11. Current: src\main\java\com\example\LearningManagementSystem\Controller\StudentController.java (61 lines)
12. Current: src\main\java\com\example\LearningManagementSystem\Controller\StudentProfileController.java (41 lines)
13. Current: src\main\java\com\example\LearningManagementSystem\DTO\StudentDTO.java (42 lines)
14. Current: src\main\java\com\example\LearningManagementSystem\DTO\StudentProfileDTO.java (84 lines)
15. Current: src\main\java\com\example\LearningManagementSystem\Entity\Category.java (23 lines)
16. Current: src\main\java\com\example\LearningManagementSystem\Entity\Course.java (50 lines)
17. Current: src\main\java\com\example\LearningManagementSystem\Entity\CourseMaterial.java (36 lines)
18. Current: src\main\java\com\example\LearningManagementSystem\Entity\Enrollment.java (52 lines)
19. Current: src\main\java\com\example\LearningManagementSystem\Entity\ErrorResponse.java (41 lines)
20. Current: src\main\java\com\example\LearningManagementSystem\Entity\Instructor.java (58 lines)
21. Current: src\main\java\com\example\LearningManagementSystem\Entity\Student.java (76 lines)
22. Current: src\main\java\com\example\LearningManagementSystem\Entity\StudentProfile.java (46 lines)
23. Current: src\main\java\com\example\LearningManagementSystem\Exception\AlreadyEnrolledException.java (10 lines)
24. Current:
src\main\java\com\example\LearningManagementSystem\Exception\CourseMaterialNotFoundException.java (8 lines)
25. Current: src\main\java\com\example\LearningManagementSystem\Exception\CourseNotFoundException.java (11 lines)
26. Current: src\main\java\com\example\LearningManagementSystem\Exception\DuplicateResourceException.java (8 lines)

27. Current: src\main\java\com\example\LearningManagementSystem\Exception\ProfileAlreadySetException.java (8 lines)

28. Current: src\main\java\com\example\LearningManagementSystem\Exception\ProfileNotFoundException.java (8 lines)

29. Current: src\main\java\com\example\LearningManagementSystem\Exception\ResourceNotFoundException.java (11 lines)

30. Current: src\main\java\com\example\LearningManagementSystem\Exception\StudentNotFoundException.java (8 lines)

31. Current:
src\main\java\com\example\LearningManagementSystem\LearningManagementSystemApplication.java (16 lines)

32. Current: src\main\java\com\example\LearningManagementSystem\Notification\EmailNotificationService.java (14 lines)

33. Current: src\main\java\com\example\LearningManagementSystem\Notification\NotificationService.java (6 lines)

34. Current: src\main\java\com\example\LearningManagementSystem\Notification\SmsNotificationService.java (12 lines)

35. Current: src\main\java\com\example\LearningManagementSystem\Projection\CourseMaterialCount.java (7 lines)

36. Current: src\main\java\com\example\LearningManagementSystem\Repository\CategoryRepo.java (14 lines)

37. Current: src\main\java\com\example\LearningManagementSystem\Repository\CourseMaterialRepo.java (27 lines)

38. Current: src\main\java\com\example\LearningManagementSystem\Repository\CourseRepo.java (18 lines)

39. Current: src\main\java\com\example\LearningManagementSystem\Repository\EnrollmentRepo.java (17 lines)

40. Current: src\main\java\com\example\LearningManagementSystem\Repository\InstructorRepo.java (14 lines)

41. Current: src\main\java\com\example\LearningManagementSystem\Repository\StudentProfileRepo.java (10 lines)

42. Current: src\main\java\com\example\LearningManagementSystem\Repository\StudentRepo.java (8 lines)

43. Current: src\main\java\com\example\LearningManagementSystem\Service\CacheInspectionService.java (26 lines)

44. Current: src\main\java\com\example\LearningManagementSystem\Service\CategoryService.java (55 lines)

45. Current: src\main\java\com\example\LearningManagementSystem\Service\CourseMaterialService.java (55 lines)

46. Current: src\main\java\com\example\LearningManagementSystem\Service\CourseService.java (86 lines)

47. Current: src\main\java\com\example\LearningManagementSystem\Service\EnrollmentService.java (70 lines)

48. Current: src\main\java\com\example\LearningManagementSystem\Service\InstructorService.java (59 lines)

49. Current: src\main\java\com\example\LearningManagementSystem\Service\StudentProfileService.java (57 lines)

50. Current: src\main\java\com\example\LearningManagementSystem\Service\StudentService.java (115 lines)

51. Current: src\main\resources\application.yaml (13 lines)

52. Current:

src\test\java\com\example\LearningManagementSystem\LearningManagementSystemApplicationTests.java (14 lines)

1 Parent: README.md

3 lines

```
1 # all_common_annotations_Project
2 Spring Boot annotations simplify app dev. Core: @SpringBootApplication, @Configuration, @Bean. Stereotypes:
  @Component, @Service, @Repository, @RestController. DI: @Autowired, @Qualifier. Web: @Get...
3
```

2 Current: .mvn\wrapper\maven-wrapper.properties

4 lines

```
1 wrapperVersion=3.3.4
2 distributionType=only-script
3 distributionUrl=https://repo.maven.apache.org/maven2/org/apache/maven/apache-maven/3.9.12/apache-maven-
3.9.12-bin.zip
4
```

```

1  <?xml version="1.0" encoding="UTF-8"?>
2  <project xmlns="http://maven.apache.org/POM/4.0.0" xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
3      xsi:schemaLocation="http://maven.apache.org/POM/4.0.0 https://maven.apache.org/xsd/maven-4.0.0.xsd">
4      <modelVersion>4.0.0</modelVersion>
5      <parent>
6          <groupId>org.springframework.boot</groupId>
7          <artifactId>spring-boot-starter-parent</artifactId>
8          <version>4.0.3</version>
9          <relativePath/> <!-- lookup parent from repository -->
10     </parent>
11     <groupId>com.example</groupId>
12     <artifactId>demo</artifactId>
13     <version>0.0.1-SNAPSHOT</version>
14     <name/>
15     <description/>
16     <url/>
17     <licenses>
18         <license/>
19     </licenses>
20     <developers>
21         <developer/>
22     </developers>
23     <scm>
24         <connection/>
25         <developerConnection/>
26         <tag/>
27         <url/>
28     </scm>
29     <properties>
30         <java.version>21</java.version>
31     </properties>
32     <dependencies>
33         <dependency>
34             <groupId>org.springframework.boot</groupId>
35             <artifactId>spring-boot-starter-cache</artifactId>
36         </dependency>
37         <dependency>
38             <groupId>org.springframework.boot</groupId>
39             <artifactId>spring-boot-starter-data-jpa</artifactId>
40         </dependency>
41         <dependency>
42             <groupId>org.springframework.boot</groupId>
43             <artifactId>spring-boot-starter-validation</artifactId>
44         </dependency>
45         <dependency>
46             <groupId>org.springframework.boot</groupId>
47             <artifactId>spring-boot-starter-webmvc</artifactId>
48         </dependency>
49
50         <dependency>
51             <groupId>org.springframework.boot</groupId>
52             <artifactId>spring-boot-devtools</artifactId>
53             <scope>runtime</scope>
54             <optional>true</optional>
55         </dependency>
56         <dependency>
57             <groupId>com.mysql</groupId>
58             <artifactId>mysql-connector-j</artifactId>
59             <scope>runtime</scope>
60         </dependency>
61         <dependency>
62             <groupId>org.springframework.boot</groupId>
63             <artifactId>spring-boot-starter-cache-test</artifactId>
64             <scope>test</scope>
65         </dependency>
66         <!-- iText PDF -->
67         <dependency>
68             <groupId>com.itextpdf</groupId>
69             <artifactId>itext7-core</artifactId>
70             <version>7.2.5</version>
71             <type>pom</type>
72         </dependency>

```

```
73     <dependency>
74         <groupId>org.springframework.boot</groupId>
75         <artifactId>spring-boot-starter-data-jpa-test</artifactId>
76         <scope>test</scope>
77     </dependency>
78     <dependency>
79         <groupId>org.springframework.boot</groupId>
80         <artifactId>spring-boot-starter-validation-test</artifactId>
81         <scope>test</scope>
82     </dependency>
83     <dependency>
84         <groupId>org.springframework.boot</groupId>
85         <artifactId>spring-boot-starter-webmvc-test</artifactId>
86         <scope>test</scope>
87     </dependency>
88 </dependencies>
89
90 <build>
91     <plugins>
92         <plugin>
93             <groupId>org.springframework.boot</groupId>
94             <artifactId>spring-boot-maven-plugin</artifactId>
95         </plugin>
96     </plugins>
97 </build>
98
99 </project>
```



```

1 package com.example.LearningManagementSystem.code;
2
3 import com.itextpdf.kernel.colors.ColorConstants;
4 import com.itextpdf.kernel.colors.DeviceRgb;
5 import com.itextpdf.kernel.font.PdfFont;
6 import com.itextpdf.kernel.font.PdfFontFactory;
7 import com.itextpdf.kernel.geom.PageSize;
8 import com.itextpdf.kernel.pdf.PdfDocument;
9 import com.itextpdf.kernel.pdf.PdfWriter;
10 import com.itextpdf.layout.Document;
11 import com.itextpdf.layout.element.AreaBreak;
12 import com.itextpdf.layout.element.Cell;
13 import com.itextpdf.layout.element.Paragraph;
14 import com.itextpdf.layout.element.Table;
15 import com.itextpdf.layout.element.Text;
16 import com.itextpdf.layout.properties.AreaBreakType;
17 import com.itextpdf.layout.properties.UnitValue;
18 import com.itextpdf.io.font.constants.StandardFonts;
19
20 import java.io.File;
21 import java.io.IOException;
22 import java.nio.file.*;
23 import java.nio.file.attribute.BasicFileAttributes;
24 import java.time.LocalDateTime;
25 import java.time.format.DateTimeFormatter;
26 import java.util.*;
27
28 /**
29  * CodeToPdf - Extracts all source code files from the current, parent,
30  * and immediate child directories into a formatted PDF document.
31  *
32  * Dependencies (add to your pom.xml or build.gradle):
33  *   com.itextpdf:itext7-core:7.2.5 (or itext7 bundle)
34  *
35  * Compile:
36  *   javac -cp itext7-core-7.2.5.jar:. CodeToPdf.java
37  *
38  * Run:
39  *   java -cp itext7-core-7.2.5.jar:. CodeToPdf
40  *
41  * Or with Maven:
42  *   mvn compile exec:java -Dexec.mainClass=CodeToPdf
43  */
44 public class CodeToPdf {
45
46     // Configuration
47
48     /** Output PDF filename */
49     private static final String OUTPUT_FILE = "code_export.pdf";
50
51     /** Maximum file size to include (skip very large generated files) */
52     private static final long MAX_FILE_SIZE_BYTES = 500_000; // 500 KB
53
54     /** Source code file extensions to include */
55     private static final Set<String> CODE_EXTENSIONS = new LinkedHashSet<>(Arrays.asList(
56         // JVM languages
57         ".java", ".kt", ".groovy", ".scala", ".clj",
58         // Web
59         ".js", ".ts", ".jsx", ".tsx", ".html", ".css", ".scss", ".less",
60         // Python / Ruby / PHP
61         ".py", ".rb", ".php",
62         // Systems / low-level
63         ".c", ".cpp", ".cc", ".h", ".hpp", ".rs", ".go", ".swift",
64         // Data / config
65         ".xml", ".json", ".yaml", ".yml", ".toml", ".properties", ".env",
66         // Shell / scripts
67         ".sh", ".bash", ".zsh", ".bat", ".ps1",
68         // Build
69         ".gradle", ".cmake", "Makefile", "Dockerfile",
70         // Other
71         ".sql", ".md", ".txt"

```

```

72     });
73
74     /** Directories to skip entirely */
75     private static final Set<String> SKIP_DIRS = new HashSet<>(Arrays.asList(
76         ".git", ".svn", ".hg", "node_modules", "target", "build",
77         "out", ".idea", ".vscode", "__pycache__", ".gradle", "dist", "vendor"
78     ));
79
80     // Colors
81     private static final DeviceRgb COLOR_HEADER_BG = new DeviceRgb(0x18, 0x5F, 0xA5); // blue
82     private static final DeviceRgb COLOR_HEADER_TEXT = new DeviceRgb(0xFF, 0xFF, 0xFF);
83     private static final DeviceRgb COLOR_FILE_TITLE = new DeviceRgb(0x18, 0x5F, 0xA5);
84     private static final DeviceRgb COLOR_CODE_BG = new DeviceRgb(0xF7, 0xF7, 0xF7);
85     private static final DeviceRgb COLOR_LINE_NUM = new DeviceRgb(0x99, 0x99, 0x99);
86     private static final DeviceRgb COLOR_TOC_HEADER = new DeviceRgb(0x18, 0x5F, 0xA5);
87
88     // Font sizes
89     private static final float FONT_TITLE = 20f;
90     private static final float FONT_SECTION = 13f;
91     private static final float FONT_FILE = 11f;
92     private static final float FONT_CODE = 7.5f;
93     private static final float FONT_TOC = 10f;
94
95     // State
96     private final List<FileEntry> fileEntries = new ArrayList<>();
97     private int totalLines = 0;
98
99     //
100
101     public static void main(String[] args) throws Exception {
102         new CodeToPdf().run();
103     }
104
105     private void run() throws Exception {
106         Path currentDir = Paths.get("").toAbsolutePath();
107         Path parentDir = currentDir.getParent();
108
109         System.out.println("=== Code to PDF Extractor ===");
110         System.out.println("Current directory : " + currentDir);
111         System.out.println("Parent directory : " + (parentDir != null ? parentDir : "(none)"));
112         System.out.println();
113
114         // 1. Collect files
115
116         // Parent directory (top-level files only, not recursive)
117         if (parentDir != null && Files.exists(parentDir)) {
118             collectFiles(parentDir, false, "Parent");
119         }
120
121         // Current directory (recursive – includes all child subdirectories)
122         collectFiles(currentDir, true, "Current");
123
124         System.out.printf("%nFound %d code file(s) with %,d total lines.%n",
125             fileEntries.size(), totalLines);
126
127         if (fileEntries.isEmpty()) {
128             System.out.println("No code files found. Exiting.");
129             return;
130         }
131
132         // 2. Generate PDF
133         generatePdf(currentDir);
134
135         System.out.println("PDF saved to: " + Paths.get(OUTPUT_FILE).toAbsolutePath());
136     }
137
138     // File Collection
139
140     private void collectFiles(Path root, boolean recursive, String scope) throws IOException {
141         if (!Files.exists(root)) return;
142
143         Files.walkFileTree(root, new SimpleFileVisitor<>() {
144             @Override
145             public FileVisitResult preVisitDirectory(Path dir, BasicFileAttributes attrs) {
146                 String name = dir.getFileName() == null ? "" : dir.getFileName().toString();
147                 // Never recurse into skip-dirs; always allow the root itself

```

```

148         if (!dir.equals(root) && SKIP_DIRS.contains(name)) {
149             return FileVisitResult.SKIP_SUBTREE;
150         }
151         // If not recursive, skip any subdirectory of root
152         if (!recursive && !dir.equals(root)) {
153             return FileVisitResult.SKIP_SUBTREE;
154         }
155         return FileVisitResult.CONTINUE;
156     }
157
158     @Override
159     public FileVisitResult visitFile(Path file, BasicFileAttributes attrs) throws IOException {
160         if (isCodeFile(file) && attrs.size() <= MAX_FILE_SIZE_BYTES) {
161             String content = Files.readString(file);
162             int lines = content.split("\n", -1).length;
163             String relativePath = scope + ": " + root.relativize(file);
164             fileEntries.add(new FileEntry(relativePath, file, content, lines));
165             totalLines += lines;
166             System.out.println("  " + relativePath + " (" + lines + " lines)");
167         }
168         return FileVisitResult.CONTINUE;
169     }
170 });
171 }
172
173 private boolean isCodeFile(Path file) {
174     String name = file.getFileName().toString();
175     // Check for exact filename match (e.g. "Makefile", "Dockerfile")
176     if (CODE_EXTENSIONS.contains(name)) return true;
177     // Check extension
178     int dot = name.lastIndexOf('.');
179     if (dot >= 0) {
180         return CODE_EXTENSIONS.contains(name.substring(dot));
181     }
182     return false;
183 }
184
185 // PDF Generation
186
187 private void generatePdf(Path currentDir) throws IOException {
188     PdfWriter writer = new PdfWriter(OUTPUT_FILE);
189     PdfDocument pdfDoc = new PdfDocument(writer);
190     Document doc = new Document(pdfDoc, PageSize.A4);
191     doc.setMargins(40, 40, 40, 40);
192
193     PdfFont sansFont = PdfFontFactory.createFont(StandardFonts.HELVETICA);
194     PdfFont boldFont = PdfFontFactory.createFont(StandardFonts.HELVETICA_BOLD);
195     PdfFont monoFont = PdfFontFactory.createFont(StandardFonts.COURIER);
196
197     // Cover page
198     addCoverPage(doc, sansFont, boldFont, currentDir);
199     doc.add(new AreaBreak(AreaBreakType.NEXT_PAGE));
200
201     // Table of contents
202     addTableOfContents(doc, sansFont, boldFont);
203     doc.add(new AreaBreak(AreaBreakType.NEXT_PAGE));
204
205     // Code sections
206     for (int i = 0; i < fileEntries.size(); i++) {
207         FileEntry entry = fileEntries.get(i);
208         addCodeSection(doc, entry, i + 1, sansFont, boldFont, monoFont);
209         if (i < fileEntries.size() - 1) {
210             doc.add(new AreaBreak(AreaBreakType.NEXT_PAGE));
211         }
212     }
213
214     doc.close();
215 }
216
217 private void addCoverPage(Document doc, PdfFont sansFont, PdfFont boldFont, Path dir)
218     throws IOException {
219
220     doc.add(new Paragraph("\n\n\n"));
221
222     // Big title
223     doc.add(new Paragraph("Code Export"))

```

```

224         .setFont(boldFont).setFontSize(FONT_TITLE + 10)
225         .setFillColor(COLOR_FILE_TITLE)
226         .setMarginBottom(6));
227
228     doc.add(new Paragraph("Source Code PDF Report")
229         .setFont(sansFont).setFontSize(FONT_SECTION)
230         .setFillColor(ColorConstants.DARK_GRAY)
231         .setMarginBottom(24));
232
233     // Stats table
234     Table stats = new Table(UnitValue.createPercentArray(new float[]{40, 60}))
235         .setWidth(UnitValue.createPercentValue(80));
236     addStatRow(stats, "Generated",      LocalDateTime.now()
237         .format(DateTimeFormatter.ofPattern("yyyy-MM-dd HH:mm")), sansFont, boldFont);
238     addStatRow(stats, "Root directory", dir.toString(), sansFont, boldFont);
239     addStatRow(stats, "Files found",   String.valueOf(fileEntries.size()), sansFont, boldFont);
240     addStatRow(stats, "Total lines",   String.format("%,d", totalLines), sansFont, boldFont);
241     doc.add(stats);
242 }
243
244 private void addStatRow(Table t, String label, String value,
245     PdfFont sans, PdfFont bold) {
246     t.addCell(new Cell().add(new Paragraph(label)
247         .setFont(bold).setFontSize(10)
248         .setFillColor(ColorConstants.DARK_GRAY))
249         .setBorder(null).setPaddingBottom(6));
250     t.addCell(new Cell().add(new Paragraph(value)
251         .setFont(sans).setFontSize(10))
252         .setBorder(null).setPaddingBottom(6));
253 }
254
255 private void addTableOfContents(Document doc, PdfFont sansFont, PdfFont boldFont)
256     throws IOException {
257     doc.add(new Paragraph("Table of Contents")
258         .setFont(boldFont).setFontSize(FONT_SECTION + 2)
259         .setFillColor(COLOR_TOC_HEADER)
260         .setMarginBottom(12));
261
262     for (int i = 0; i < fileEntries.size(); i++) {
263         FileEntry entry = fileEntries.get(i);
264         String label = String.format("%d.  %s  (%,d lines)",
265             i + 1, entry.relativePath, entry.lineCount);
266         doc.add(new Paragraph(label)
267             .setFont(sansFont).setFontSize(FONT_TOC)
268             .setMarginBottom(3));
269     }
270 }
271
272 private void addCodeSection(Document doc, FileEntry entry, int index,
273     PdfFont sansFont, PdfFont boldFont, PdfFont monoFont)
274     throws IOException {
275
276     // Section header bar
277     Table header = new Table(UnitValue.createPercentArray(new float[]{5, 75, 20}))
278         .setWidth(UnitValue.createPercentValue(100))
279         .setMarginBottom(8);
280
281     header.addCell(new Cell().add(new Paragraph(String.valueOf(index))
282         .setFont(boldFont).setFontSize(FONT_FILE)
283         .setFillColor(COLOR_HEADER_TEXT))
284         .setBackgroundColor(COLOR_HEADER_BG)
285         .setBorder(null).setPadding(6));
286
287     header.addCell(new Cell().add(new Paragraph(entry.relativePath)
288         .setFont(boldFont).setFontSize(FONT_FILE)
289         .setFillColor(COLOR_HEADER_TEXT))
290         .setBackgroundColor(COLOR_HEADER_BG)
291         .setBorder(null).setPadding(6));
292
293     header.addCell(new Cell().add(new Paragraph(entry.lineCount + " lines")
294         .setFont(sansFont).setFontSize(FONT_FILE - 1)
295         .setFillColor(COLOR_HEADER_TEXT))
296         .setBackgroundColor(COLOR_HEADER_BG)
297         .setBorder(null).setPadding(6));
298
299     doc.add(header);

```

```

300
301 // Code block with line numbers
302 String[] lines = entry.content.split("\n", -1);
303 int padWidth = String.valueOf(lines.length).length();
304
305 for (int ln = 0; ln < lines.length; ln++) {
306     String lineNum = String.format("%" + padWidth + "d", ln + 1);
307     String codeLine = lines[ln]
308         .replace("\t", "    ") // expand tabs
309         .replace("\r", "");     // strip CR
310
311     // Truncate very long lines
312     if (codeLine.length() > 200) {
313         codeLine = codeLine.substring(0, 197) + "...";
314     }
315
316     Text numText = new Text(lineNum + " ").setFont(monoFont)
317         .setFontSize(FONT_CODE).setFontColor(COLOR_LINE_NUM);
318     Text codeText = new Text(codeLine).setFont(monoFont)
319         .setFontSize(FONT_CODE).setFontColor(ColorConstants.BLACK);
320
321     Paragraph p = new Paragraph().add(numText).add(codeText)
322         .setBackgroundColor(ln % 2 == 0 ? COLOR_CODE_BG : ColorConstants.WHITE)
323         .setMargin(0).setPaddingLeft(4).setPaddingRight(4)
324         .setMultipliedLeading(1.3f)
325         .setFixedLeading(FONT_CODE + 2.5f);
326
327     doc.add(p);
328 }
329 }
330
331 // Data class
332
333 private static class FileEntry {
334     final String relativePath;
335     final Path absolutePath;
336     final String content;
337     final int lineCount;
338
339     FileEntry(String relativePath, Path absolutePath, String content, int lineCount) {
340         this.relativePath = relativePath;
341         this.absolutePath = absolutePath;
342         this.content = content;
343         this.lineCount = lineCount;
344     }
345 }
346 }
347

```

```
1 package com.example.LearningManagementSystem.Controller;
2
3 import com.example.LearningManagementSystem.Entity.Category;
4 import com.example.LearningManagementSystem.Service.CategoryService;
5 import jakarta.validation.Valid;
6 import org.springframework.http.HttpStatus;
7 import org.springframework.http.ResponseEntity;
8 import org.springframework.web.bind.annotation.*;
9
10 import java.util.List;
11
12 @RestController
13 @RequestMapping("/api/categories")
14 public class CategoryController {
15
16     private final CategoryService categoryService;
17
18     public CategoryController(CategoryService categoryService) {
19         this.categoryService = categoryService;
20     }
21
22     @GetMapping
23     public ResponseEntity<List<Category>> getAllCategories() {
24         return ResponseEntity.ok(categoryService.getAllCategories());
25     }
26
27     @GetMapping("/{id}")
28     public ResponseEntity<Category> getCategoryById(@PathVariable Long id) {
29         return ResponseEntity.ok(categoryService.getCategoryById(id));
30     }
31
32     @PostMapping
33     public ResponseEntity<Category> createCategory(@Valid @RequestBody Category category) {
34         return new ResponseEntity<>(categoryService.createCategory(category), HttpStatus.CREATED);
35     }
36
37     @PutMapping("/{id}")
38     public ResponseEntity<Category> updateCategory(@PathVariable Long id, @Valid @RequestBody Category
category) {
39         return ResponseEntity.ok(categoryService.updateCategory(id, category));
40     }
41
42     @DeleteMapping("/{id}")
43     public ResponseEntity<Void> deleteCategory(@PathVariable Long id) {
44         categoryService.deleteCategory(id);
45         return ResponseEntity.noContent().build();
46     }
47 }
```

```
1 package com.example.LearningManagementSystem.Controller;
2
3 import com.example.LearningManagementSystem.Entity.Course;
4 import com.example.LearningManagementSystem.Service.CourseService;
5 import jakarta.validation.Valid;
6 import org.springframework.http.HttpStatus;
7 import org.springframework.http.ResponseEntity;
8 import org.springframework.web.bind.annotation.*;
9
10 import java.util.List;
11
12 @RestController
13 @RequestMapping("/api/courses")
14 public class CourseController {
15
16     private final CourseService courseService;
17
18     public CourseController(CourseService courseService) {
19         this.courseService = courseService;
20     }
21
22     @GetMapping
23     public ResponseEntity<List<Course>> getAllCourses() {
24         return ResponseEntity.ok(courseService.getAllCourses());
25     }
26
27     @GetMapping("/{id}")
28     public ResponseEntity<Course> getCourseById(@PathVariable Long id) {
29         return ResponseEntity.ok(courseService.getCourseById(id));
30     }
31
32     @GetMapping("/instructor/{instructorId}")
33     public ResponseEntity<List<Course>> getCoursesByInstructor(@PathVariable Long instructorId) {
34         return ResponseEntity.ok(courseService.getCoursesByInstructor(instructorId));
35     }
36
37     @GetMapping("/category/{categoryId}")
38     public ResponseEntity<List<Course>> getCoursesByCategory(@PathVariable Long categoryId) {
39         return ResponseEntity.ok(courseService.getCoursesByCategory(categoryId));
40     }
41
42     @GetMapping("/search")
43     public ResponseEntity<List<Course>> searchCourses(@RequestParam String keyword) {
44         return ResponseEntity.ok(courseService.searchCourses(keyword));
45     }
46
47     @PostMapping
48     public ResponseEntity<Course> createCourse(
49         @Valid @RequestBody Course course,
50         @RequestParam Long instructorId,
51         @RequestParam Long categoryId) {
52         return new ResponseEntity<>(courseService.createCourse(course, instructorId, categoryId),
53 HttpStatus.CREATED);
54     }
55
56     @PutMapping("/{id}")
57     public ResponseEntity<Course> updateCourse(
58         @PathVariable Long id,
59         @Valid @RequestBody Course course,
60         @RequestParam Long instructorId,
61         @RequestParam Long categoryId) {
62         return ResponseEntity.ok(courseService.updateCourse(id, course, instructorId, categoryId));
63     }
64
65     @DeleteMapping("/{id}")
66     public ResponseEntity<Void> deleteCourse(@PathVariable Long id) {
67         courseService.deleteCourse(id);
68         return ResponseEntity.noContent().build();
69     }
```

7 Current:

src\main\java\com\example\LearningManagementSystem\Controller\CourseMaterialController.java

```
1 package com.example.LearningManagementSystem.Controller;
2
3 import com.example.LearningManagementSystem.Entity.CourseMaterial;
4 import com.example.LearningManagementSystem.Service.CourseMaterialService;
5 import jakarta.validation.Valid;
6 import org.springframework.http.HttpStatus;
7 import org.springframework.http.ResponseEntity;
8 import org.springframework.web.bind.annotation.*;
9
10 import java.util.List;
11
12 @RestController
13 @RequestMapping("/api/CourseMaterials")
14 public class CourseMaterialController {
15
16     private final CourseMaterialService courseMaterialService;
17
18     public CourseMaterialController(CourseMaterialService courseMaterialService) {
19         this.courseMaterialService = courseMaterialService;
20     }
21
22     @GetMapping
23     public List<CourseMaterial> getAllMaterials() {
24         return courseMaterialService.getAllMaterials();
25     }
26
27     @PostMapping
28     public ResponseEntity<CourseMaterial> addMaterial(
29         @Valid @RequestBody CourseMaterial material,
30         @RequestParam String course_name) {
31         CourseMaterial addedMaterial = courseMaterialService.addMaterial(material, course_name);
32         // BUG FIX: was HttpStatus.OK (200), should be CREATED (201) for a POST
33         return new ResponseEntity<>(addedMaterial, HttpStatus.CREATED);
34     }
35
36     @GetMapping("/{id}")
37     public ResponseEntity<List<CourseMaterial>> getCourseMaterialById(@PathVariable long id) {
38         List<CourseMaterial> courseMaterials = courseMaterialService.getMaterialById(id);
39         return new ResponseEntity<>(courseMaterials, HttpStatus.OK);
40     }
41
42     @DeleteMapping("/{id}")
43     public ResponseEntity<String> deleteCourseMaterial(@PathVariable int id) {
44         courseMaterialService.deleteMaterial(id);
45         return ResponseEntity.status(HttpStatus.OK)
46             .body("Course material : " + id + " successfully deleted");
47     }
48
49     @GetMapping("/getCount")
50     public ResponseEntity<List<?>> getCourseMaterialCount() {
51         return courseMaterialService.getMaterialCount();
52     }
53 }
```



```
1 package com.example.LearningManagementSystem.Controller;
2
3 import com.example.LearningManagementSystem.Entity.Enrollment;
4 import com.example.LearningManagementSystem.Service.EnrollmentService;
5 import org.springframework.http.HttpStatus;
6 import org.springframework.http.ResponseEntity;
7 import org.springframework.web.bind.annotation.*;
8
9 import java.util.List;
10
11 @RestController
12 @RequestMapping("/api/enrollments")
13 public class EnrollmentController {
14
15     private final EnrollmentService enrollmentService;
16
17     public EnrollmentController(EnrollmentService enrollmentService) {
18         this.enrollmentService = enrollmentService;
19     }
20
21     @GetMapping
22     public ResponseEntity<List<Enrollment>> getAllEnrollments() {
23         return ResponseEntity.ok(enrollmentService.getAllEnrollments());
24     }
25
26     @GetMapping("/{id}")
27     public ResponseEntity<Enrollment> getEnrollmentById(@PathVariable Long id) {
28         return ResponseEntity.ok(enrollmentService.getEnrollmentById(id));
29     }
30
31     @GetMapping("/student/{studentId}")
32     public ResponseEntity<List<Enrollment>> getEnrollmentsByStudent(@PathVariable Long studentId) {
33         return ResponseEntity.ok(enrollmentService.getEnrollmentsByStudent(studentId));
34     }
35
36     @GetMapping("/course/{courseId}")
37     public ResponseEntity<List<Enrollment>> getEnrollmentsByCourse(@PathVariable Long courseId) {
38         return ResponseEntity.ok(enrollmentService.getEnrollmentsByCourse(courseId));
39     }
40
41     @PostMapping
42     public ResponseEntity<Enrollment> enroll(
43         @RequestParam Long studentId,
44         @RequestParam Long courseId) {
45         return new ResponseEntity<>(enrollmentService.enroll(studentId, courseId), HttpStatus.CREATED);
46     }
47
48     @DeleteMapping("/{id}")
49     public ResponseEntity<Void> unenroll(@PathVariable Long id) {
50         enrollmentService.unenroll(id);
51         return ResponseEntity.noContent().build();
52     }
53 }
54
```

```

1 package com.example.LearningManagementSystem.Controller;
2
3 import com.example.LearningManagementSystem.Exception.*;
4 import org.springframework.http.HttpStatus;
5 import org.springframework.http.ResponseEntity;
6 import org.springframework.validation.FieldError;
7 import org.springframework.web.bind.MethodArgumentNotValidException;
8 import org.springframework.web.bind.annotation.ExceptionHandler;
9 import org.springframework.web.bind.annotation.RestControllerAdvice;
10 import org.springframework.web.context.request.WebRequest;
11
12 import java.time.LocalDateTime;
13 import java.util.HashMap;
14 import java.util.LinkedHashMap;
15 import java.util.List;
16 import java.util.Map;
17 import java.util.stream.Collectors;
18
19 @RestControllerAdvice
20 public class GlobalExceptionHandler {
21
22     private Map<String, Object> buildError(HttpStatus status, String message, WebRequest request) {
23         Map<String, Object> body = new LinkedHashMap<>();
24         body.put("timestamp", LocalDateTime.now());
25         body.put("status", status.value());
26         body.put("error", status.getReasonPhrase());
27         body.put("message", message);
28         body.put("path", request.getDescription(false).replace("uri=", ""));
29         return body;
30     }
31
32     // Handles @Valid failures – collects ALL field errors into a readable list
33     @ExceptionHandler(MethodArgumentNotValidException.class)
34     public ResponseEntity<Map<String, Object>> handleValidationErrors(
35         MethodArgumentNotValidException ex, WebRequest request) {
36
37         List<String> fieldErrors = ex.getBindingResult()
38             .getFieldErrors()
39             .stream()
40             .map(fe -> fe.getField() + ": " + fe.getDefaultMessage())
41             .collect(Collectors.toList());
42
43         Map<String, Object> body = new LinkedHashMap<>();
44         body.put("timestamp", LocalDateTime.now());
45         body.put("status", HttpStatus.BAD_REQUEST.value());
46         body.put("error", "Validation failed");
47         body.put("messages", fieldErrors);
48         body.put("path", request.getDescription(false).replace("uri=", ""));
49         return new ResponseEntity<>(body, HttpStatus.BAD_REQUEST);
50     }
51
52     @ExceptionHandler(StudentNotFoundException.class)
53     public ResponseEntity<Map<String, Object>> handelStudentNotFoundException(
54         StudentNotFoundException exception, WebRequest request) {
55         return new ResponseEntity<>(buildError(HttpStatus.NOT_FOUND,
56             exception.getMessage(), request), HttpStatus.NOT_FOUND);
57     }
58
59     @ExceptionHandler(ProfileAlreadySetException.class)
60     public ResponseEntity<Map<String, Object>> handelProfileAlreadySetException(
61         ProfileAlreadySetException exception, WebRequest request) {
62         return new ResponseEntity<>(buildError(HttpStatus.BAD_REQUEST,
63             exception.getMessage(), request), HttpStatus.BAD_REQUEST);
64     }
65
66     @ExceptionHandler(ProfileNotFoundException.class)
67     public ResponseEntity<Map<String, Object>> handelProfileNotFoundException(
68         ProfileNotFoundException exception, WebRequest request) {
69         return new ResponseEntity<>(buildError(HttpStatus.NOT_FOUND,
70             exception.getMessage(), request), HttpStatus.NOT_FOUND);
71     }

```

```

72
73     @ExceptionHandler(CourseMaterialNotFoundException.class)
74     public ResponseEntity<Map<String, Object>> handelCourseMaterialNotFoundException(
75         CourseMaterialNotFoundException exception, WebRequest request) {
76         return new ResponseEntity<>(buildError(HttpStatus.NOT_FOUND,
77             exception.getMessage(), request), HttpStatus.NOT_FOUND);
78     }
79
80     @ExceptionHandler(CourseNotFoundException.class)
81     public ResponseEntity<Map<String, Object>> handelCourseNotFoundException(
82         CourseNotFoundException exception, WebRequest request) {
83         return new ResponseEntity<>(buildError(HttpStatus.NOT_FOUND,
84             exception.getMessage(), request), HttpStatus.NOT_FOUND);
85     }
86
87     @ExceptionHandler(ResourceNotFoundException.class)
88     public ResponseEntity<Map<String, Object>> handleResourceNotFound(
89         ResourceNotFoundException ex, WebRequest request) {
90         return new ResponseEntity<>(buildError(HttpStatus.NOT_FOUND,
91             ex.getMessage(), request), HttpStatus.NOT_FOUND);
92     }
93
94     @ExceptionHandler(DuplicateResourceException.class)
95     public ResponseEntity<Map<String, Object>> handleDuplicateResource(
96         DuplicateResourceException ex, WebRequest request) {
97         return new ResponseEntity<>(buildError(HttpStatus.CONFLICT,
98             ex.getMessage(), request), HttpStatus.CONFLICT);
99     }
100
101     @ExceptionHandler(AlreadyEnrolledException.class)
102     public ResponseEntity<Map<String, Object>> handleAlreadyEnrolled(
103         AlreadyEnrolledException ex, WebRequest request) {
104         return new ResponseEntity<>(buildError(HttpStatus.CONFLICT,
105             ex.getMessage(), request), HttpStatus.CONFLICT);
106     }
107
108     @ExceptionHandler(IllegalArgumentException.class)
109     public ResponseEntity<Map<String, Object>> handleIllegalArgument(
110         IllegalArgumentException ex, WebRequest request) {
111         return new ResponseEntity<>(buildError(HttpStatus.BAD_REQUEST,
112             ex.getMessage(), request), HttpStatus.BAD_REQUEST);
113     }
114
115
116     @ExceptionHandler(Exception.class)
117     public ResponseEntity<Map<String, Object>> handleGeneral(
118         Exception ex, WebRequest request) {
119         return new ResponseEntity<>(buildError(HttpStatus.INTERNAL_SERVER_ERROR,
120             "An unexpected error occurred.", request), HttpStatus.INTERNAL_SERVER_ERROR);
121     }
122 }

```

```
1 package com.example.LearningManagementSystem.Controller;
2
3 import com.example.LearningManagementSystem.Entity.Instructor;
4 import com.example.LearningManagementSystem.Service.InstructorService;
5 import jakarta.validation.Valid;
6 import org.springframework.http.HttpStatus;
7 import org.springframework.http.ResponseEntity;
8 import org.springframework.web.bind.annotation.*;
9
10 import java.util.List;
11
12 @RestController
13 @RequestMapping("/api/instructors")
14 public class InstructorController {
15
16     private final InstructorService instructorService;
17
18     public InstructorController(InstructorService instructorService) {
19         this.instructorService = instructorService;
20     }
21
22     @GetMapping
23     public ResponseEntity<List<Instructor>> getAllInstructors() {
24         return ResponseEntity.ok(instructorService.getAllInstructors());
25     }
26
27     @GetMapping("/{id}")
28     public ResponseEntity<Instructor> getInstructorById(@PathVariable Long id) {
29         return ResponseEntity.ok(instructorService.getInstructorById(id));
30     }
31
32     @PostMapping
33     public ResponseEntity<Instructor> createInstructor(@Valid @RequestBody Instructor instructor) {
34         return new ResponseEntity<>(instructorService.createInstructor(instructor), HttpStatus.CREATED);
35     }
36
37     @PutMapping("/{id}")
38     public ResponseEntity<Instructor> updateInstructor(@PathVariable Long id, @Valid @RequestBody Instructor
instructor) {
39         return ResponseEntity.ok(instructorService.updateInstructor(id, instructor));
40     }
41
42     @DeleteMapping("/{id}")
43     public ResponseEntity<Void> deleteInstructor(@PathVariable Long id) {
44         instructorService.deleteInstructor(id);
45         return ResponseEntity.noContent().build();
46     }
47 }
```

```
1 package com.example.LearningManagementSystem.Controller;
2
3 import com.example.LearningManagementSystem.DTO.StudentProfileDTO;
4 import com.example.LearningManagementSystem.Entity.Student;
5 import com.example.LearningManagementSystem.Service.CacheInspectionService;
6 import com.example.LearningManagementSystem.Service.StudentService;
7 import jakarta.validation.Valid;
8 import org.springframework.http.HttpStatus;
9 import org.springframework.http.ResponseEntity;
10 import org.springframework.web.bind.annotation.*;
11
12 import java.util.List;
13
14 @RestController
15 @RequestMapping("/api/students")
16 public class StudentController {
17
18     private final StudentService service;
19     private final CacheInspectionService inspectionService;
20
21     public StudentController(StudentService service, CacheInspectionService inspectionService) {
22         this.service = service;
23         this.inspectionService = inspectionService;
24     }
25
26     @GetMapping
27     public List<StudentProfileDTO> getAllStudents() {
28         return service.getAllStudents();
29     }
30
31     @PostMapping
32     public ResponseEntity<StudentProfileDTO> addStudent(@Valid @RequestBody Student student) {
33         // BUG FIX: was HttpStatus.OK (200), should be CREATED (201) for a POST
34         return ResponseEntity.status(HttpStatus.CREATED)
35             .body(service.addStudent(student));
36     }
37
38     @DeleteMapping("/{id}")
39     public ResponseEntity<String> deleteStudent(@PathVariable long id) {
40         service.deleteStudent(id);
41         return ResponseEntity.status(HttpStatus.OK)
42             .body("deleted student with id:" + id);
43     }
44
45     @PutMapping("/{id}")
46     public ResponseEntity<String> updateStudent(@PathVariable long id, @Valid @RequestBody Student student)
47     {
48         service.updateStudent(id, student);
49         return ResponseEntity.status(HttpStatus.OK)
50             .body("updated student with id:" + id);
51     }
52
53     @GetMapping("/{id}")
54     public ResponseEntity<StudentProfileDTO> getStudentById(@PathVariable int id){
55         return ResponseEntity.ok(service.getStudentById(id));
56     }
57
58     @GetMapping("/Cache")
59     public void getCache(){
60         inspectionService.printCache("Students");
61     }
```

12 Current:

src\main\java\com\example\LearningManagementSystem\Controller\StudentProfileController.java

```
1 package com.example.LearningManagementSystem.Controller;
2
3 import com.example.LearningManagementSystem.Entity.StudentProfile;
4 import com.example.LearningManagementSystem.Service.StudentProfileService;
5 import jakarta.validation.Valid;
6 import org.springframework.http.HttpStatus;
7 import org.springframework.http.ResponseEntity;
8 import org.springframework.web.bind.annotation.*;
9
10 import java.util.List;
11
12 @RestController
13 @RequestMapping("/api/studentprofiles")
14 public class StudentProfileController {
15
16     private final StudentProfileService service;
17
18     public StudentProfileController(StudentProfileService service) {
19         this.service = service;
20     }
21
22     @GetMapping
23     public List<StudentProfile> getAllprofile() {
24         return service.getAllProfile();
25     }
26
27     @PostMapping
28     public ResponseEntity<StudentProfile> addProfile(
29         @Valid @RequestBody StudentProfile studentProfile,
30         @RequestParam long id) {
31         return ResponseEntity.status(HttpStatus.CREATED)
32             .body(service.addProfile(studentProfile, id));
33     }
34
35     @DeleteMapping("/{id}")
36     public ResponseEntity<String> deleteProfile(@PathVariable long id) {
37         service.deleteProfile(id);
38         return ResponseEntity.status(HttpStatus.OK)
39             .body("Profile with id:" + id + " deleted");
40     }
41 }
```

```
1 package com.example.LearningManagementSystem.DTO;
2
3 public class StudentDTO {
4     private Long studentId;
5     private String name;
6     private String email;
7
8     public String getPhoneNumber() {
9         return phoneNumber;
10    }
11
12    public void setPhoneNumber(String phoneNumber) {
13        this.phoneNumber = phoneNumber;
14    }
15
16    public String getEmail() {
17        return email;
18    }
19
20    public void setEmail(String email) {
21        this.email = email;
22    }
23
24    public String getName() {
25        return name;
26    }
27
28    public void setName(String name) {
29        this.name = name;
30    }
31
32    public Long getStudentId() {
33        return studentId;
34    }
35
36    public void setStudentId(Long studentId) {
37        this.studentId = studentId;
38    }
39
40    private String phoneNumber;
41 }
42
```

```
1 package com.example.LearningManagementSystem.DTO;
2
3 import java.time.LocalDate;
4
5 public class StudentProfileDTO {
6     private Long studentId;
7     private String name;
8     private String email;
9     private String phoneNumber;
10    private String address;
11    private LocalDate dateOfBirth;
12    private String education;
13
14    public Long getStudentId() {
15        return studentId;
16    }
17
18    public void setStudentId(Long studentId) {
19        this.studentId = studentId;
20    }
21
22    public String getName() {
23        return name;
24    }
25
26    public void setName(String name) {
27        this.name = name;
28    }
29
30    public String getEmail() {
31        return email;
32    }
33
34    public void setEmail(String email) {
35        this.email = email;
36    }
37
38    public String getPhoneNumber() {
39        return phoneNumber;
40    }
41
42    public void setPhoneNumber(String phoneNumber) {
43        this.phoneNumber = phoneNumber;
44    }
45
46    public String getAddress() {
47        return address;
48    }
49
50    public void setAddress(String address) {
51        this.address = address;
52    }
53
54    public LocalDate getDateOfBirth() {
55        return dateOfBirth;
56    }
57
58    public void setDateOfBirth(LocalDate dateOfBirth) {
59        this.dateOfBirth = dateOfBirth;
60    }
61
62    public String getEducation() {
63        return education;
64    }
65
66    public void setEducation(String education) {
67        this.education = education;
68    }
69
70    @Override
71    public String toString() {
```



```
72         return "StudentProfileDTO{" +
73             "studentId=" + studentId +
74             ", name='" + name + '\'' +
75             ", email='" + email + '\'' +
76             ", phoneNumber='" + phoneNumber + '\'' +
77             ", address='" + address + '\'' +
78             ", dateOfBirth=" + dateOfBirth +
79             ", education='" + education + '\'' +
80             '}' ;
81     }
82
83 }
84
```

```
1 package com.example.LearningManagementSystem.Entity;
2
3 import jakarta.persistence.*;
4 import jakarta.validation.constraints.NotBlank;
5 import jakarta.validation.constraints.Size;
6
7 @Entity
8 public class Category {
9
10     @Id
11     @GeneratedValue(strategy = GenerationType.IDENTITY)
12     private Long categoryId;
13
14     @NotBlank(message = "Category name is required")
15     @Size(min = 2, max = 100, message = "Category name must be between 2 and 100 characters")
16     private String name;
17
18     public Long getCategoryId() { return categoryId; }
19     public void setCategoryId(Long categoryId) { this.categoryId = categoryId; }
20
21     public String getName() { return name; }
22     public void setName(String name) { this.name = name; }
23 }
```

```
1 package com.example.LearningManagementSystem.Entity;
2
3 import jakarta.persistence.*;
4 import jakarta.validation.constraints.NotBlank;
5 import jakarta.validation.constraints.Size;
6
7 import java.time.LocalDateTime;
8
9 @Entity
10 public class Course {
11
12     @Id
13     @GeneratedValue(strategy = GenerationType.IDENTITY)
14     private Long courseId;
15
16     @NotBlank(message = "Course name is required")
17     @Size(min = 3, max = 150, message = "Course name must be between 3 and 150 characters")
18     private String courseName;
19
20     @Size(max = 1000, message = "Description cannot exceed 1000 characters")
21     private String description;
22
23     private LocalDateTime created_at;
24
25     @ManyToOne
26     @JoinColumn(name = "instructor_id")
27     private Instructor instructor;
28
29     @ManyToOne
30     @JoinColumn(name = "category_id")
31     private Category category;
32
33     public Long getCourseId() { return courseId; }
34     public void setCourseId(Long courseId) { this.courseId = courseId; }
35
36     public String getCourseName() { return courseName; }
37     public void setCourseName(String courseName) { this.courseName = courseName; }
38
39     public String getDescription() { return description; }
40     public void setDescription(String description) { this.description = description; }
41
42     public LocalDateTime getCreated_at() { return created_at; }
43     public void setCreated_at(LocalDateTime created_at) { this.created_at = created_at; }
44
45     public Instructor getInstructor() { return instructor; }
46     public void setInstructor(Instructor instructor) { this.instructor = instructor; }
47
48     public Category getCategory() { return category; }
49     public void setCategory(Category category) { this.category = category; }
50 }
```

```
1 package com.example.LearningManagementSystem.Entity;
2
3 import jakarta.persistence.*;
4 import jakarta.validation.constraints.NotBlank;
5 import jakarta.validation.constraints.Size;
6
7 @Entity
8 public class CourseMaterial {
9
10     @Id
11     @GeneratedValue(strategy = GenerationType.IDENTITY)
12     private long id;
13
14     @NotBlank(message = "Title is required")
15     @Size(max = 200, message = "Title cannot exceed 200 characters")
16     private String title;
17
18     @NotBlank(message = "File URL is required")
19     private String file_url;
20
21     @ManyToOne
22     @JoinColumn(name = "course_id")
23     private Course course;
24
25     public long getId() { return id; }
26     public void setId(long id) { this.id = id; }
27
28     public String getTitle() { return title; }
29     public void setTitle(String title) { this.title = title; }
30
31     public String getFile_url() { return file_url; }
32     public void setFile_url(String file_url) { this.file_url = file_url; }
33
34     public Course getCourse() { return course; }
35     public void setCourse(Course course) { this.course = course; }
36 }
```

```
1 package com.example.LearningManagementSystem.Entity;
2
3 import jakarta.persistence.*;
4
5 import java.time.LocalDateTime;
6
7 @Entity
8 public class Enrollment {
9     @Id
10     @GeneratedValue(strategy = GenerationType.IDENTITY)
11     private Long enrollmentId;
12
13     @ManyToOne
14     @JoinColumn(name = "student_id")
15     private Student student;
16     @ManyToOne
17     @JoinColumn(name = "course_id")
18     private Course course;
19     private LocalDateTime enrolledDate;
20
21     public Long getEnrollmentId() {
22         return enrollmentId;
23     }
24
25     public void setEnrollmentId(Long enrollmentId) {
26         this.enrollmentId = enrollmentId;
27     }
28
29     public LocalDateTime getEnrolledDate() {
30         return enrolledDate;
31     }
32     public void setEnrolledDate(LocalDateTime enrolledDate) {
33         this.enrolledDate = enrolledDate;
34     }
35
36     public Course getCourse() {
37         return course;
38     }
39
40     public void setCourse(Course course) {
41         this.course = course;
42     }
43
44     public Student getStudent() {
45         return student;
46     }
47
48     public void setStudent(Student student) {
49         this.student = student;
50     }
51 }
52
```

```
1 package com.example.LearningManagementSystem.Entity;
2
3 import java.time.LocalDateTime;
4
5 public class ErrorResponse {
6     private LocalDateTime timestamp;
7     private String error;
8     private String message;
9
10
11     public ErrorResponse(LocalDateTime timestamp, String error, String message) {
12         this.timestamp = timestamp;
13         this.error = error;
14         this.message = message;
15     }
16
17     public LocalDateTime getTimestamp() {
18         return timestamp;
19     }
20
21     public void setTimestamp(LocalDateTime timestamp) {
22         this.timestamp = timestamp;
23     }
24
25     public String getError() {
26         return error;
27     }
28
29     public void setError(String error) {
30         this.error = error;
31     }
32
33     public String getMessage() {
34         return message;
35     }
36
37     public void setMessage(String message) {
38         this.message = message;
39     }
40 }
41
```

```
1 package com.example.LearningManagementSystem.Entity;
2
3 import com.fasterxml.jackson.annotation.JsonIgnore;
4 import jakarta.persistence.*;
5 import jakarta.validation.constraints.*;
6
7 import java.util.List;
8
9 @Entity
10 public class Instructor {
11
12     @Id
13     @GeneratedValue(strategy = GenerationType.IDENTITY)
14     private Long instructorId;
15
16     @NotBlank(message = "Name is required")
17     @Size(min = 2, max = 100, message = "Name must be between 2 and 100 characters")
18     private String name;
19
20     @NotBlank(message = "Email is required")
21     @Email(message = "Email must be a valid email address")
22     private String email;
23
24     @NotBlank(message = "Password is required")
25     @Size(min = 6, message = "Password must be at least 6 characters")
26     private String password;
27
28     private String specialization;
29
30     @Min(value = 0, message = "Experience years cannot be negative")
31     @Max(value = 60, message = "Experience years seems unrealistic")
32     private short experienceYears;
33
34     @OneToMany(mappedBy = "instructor")
35     @JsonIgnore
36     private List<Course> courses;
37
38     public Long getInstructorId() { return instructorId; }
39     public void setInstructorId(Long instructorId) { this.instructorId = instructorId; }
40
41     public String getName() { return name; }
42     public void setName(String name) { this.name = name; }
43
44     public String getEmail() { return email; }
45     public void setEmail(String email) { this.email = email; }
46
47     public String getPassword() { return password; }
48     public void setPassword(String password) { this.password = password; }
49
50     public String getSpecialization() { return specialization; }
51     public void setSpecialization(String specialization) { this.specialization = specialization; }
52
53     public short getExperienceYears() { return experienceYears; }
54     public void setExperienceYears(short experienceYears) { this.experienceYears = experienceYears; }
55
56     public List<Course> getCourses() { return courses; }
57     public void setCourses(List<Course> courses) { this.courses = courses; }
58 }
```

```
1 package com.example.LearningManagementSystem.Entity;
2
3 import jakarta.persistence.*;
4 import jakarta.validation.constraints.Email;
5 import jakarta.validation.constraints.Pattern;
6
7 @Entity
8 public class Student {
9     @Id
10     @GeneratedValue(strategy = GenerationType.IDENTITY)
11     private Long studentId;
12
13     private String name;
14
15     @Column(unique = true)
16     @Email(message = "Must be a valid email")
17     //@Pattern(regexp = "[a-zA-Z0-9._%+-]+@[a-zA-Z0-9.-]+\.[a-zA-Z]{2,}$",
18             //message = "Invalid email format")
19     private String email;
20
21     private String password;
22     @Pattern(regexp = "\\d{10}", message = "Phone number must be 10 digits")
23     private String phoneNumber;
24
25     @OneToOne(mappedBy = "student", cascade = CascadeType.ALL)
26     private StudentProfile profile;
27
28     public Long getStudentId() {
29         return studentId;
30     }
31
32     public void setStudentId(Long studentId) {
33         this.studentId = studentId;
34     }
35
36     public String getEmail() {
37         return email;
38     }
39
40     public void setEmail(String email) {
41         this.email = email;
42     }
43
44     public String getName() {
45         return name;
46     }
47
48     public void setName(String name) {
49         this.name = name;
50     }
51
52     public String getPassword() {
53         return password;
54     }
55
56     public void setPassword(String password) {
57         this.password = password;
58     }
59
60     public String getPhoneNumber() {
61         return phoneNumber;
62     }
63
64     public void setPhoneNumber(String phoneNumber) {
65         this.phoneNumber = phoneNumber;
66     }
67
68     public StudentProfile getProfile() {
69         return profile;
70     }
71 }
```



```
72     public void setProfile(StudentProfile profile) {  
73         this.profile = profile;  
74     }  
75 }  
76
```

```
1 package com.example.LearningManagementSystem.Entity;
2
3 import com.fasterxml.jackson.annotation.JsonIgnore;
4 import jakarta.persistence.*;
5 import jakarta.validation.constraints.Past;
6 import jakarta.validation.constraints.Size;
7
8 import java.time.LocalDate;
9
10 @Entity
11 public class StudentProfile {
12
13     @Id
14     @GeneratedValue(strategy = GenerationType.IDENTITY)
15     private Long profileId;
16
17     @Size(max = 300, message = "Address cannot exceed 300 characters")
18     private String address;
19
20     @Column(name = "date_of_birth")
21     @Past(message = "Date of birth must be in the past")
22     private LocalDate dateOfBirth;
23
24     @Size(max = 200, message = "Education field cannot exceed 200 characters")
25     private String education;
26
27     @OneToOne
28     @JoinColumn(name = "student_id")
29     @JsonIgnore
30     private Student student;
31
32     public Long getProfileId() { return profileId; }
33     public void setProfileId(Long profileId) { this.profileId = profileId; }
34
35     public String getAddress() { return address; }
36     public void setAddress(String address) { this.address = address; }
37
38     public LocalDate getDateOfBirth() { return dateOfBirth; }
39     public void setDateOfBirth(LocalDate dateOfBirth) { this.dateOfBirth = dateOfBirth; }
40
41     public String getEducation() { return education; }
42     public void setEducation(String education) { this.education = education; }
43
44     public Student getStudent() { return student; }
45     public void setStudent(Student student) { this.student = student; }
46 }
```

23 Current:

src\main\java\com\example\LearningManagementSystem\Exception\AlreadyEnrolledException.java

```
1 package com.example.LearningManagementSystem.Exception;
2
3 import org.springframework.web.bind.annotation.ExceptionHandler;
4
5 public class AlreadyEnrolledException extends RuntimeException {
6     public AlreadyEnrolledException(String message) {
7         super(message);
8     }
9 }
10
```

24 Current:

src\main\java\com\example\LearningManagementSystem\Exception\CourseMaterialNotFoundExcept

```
1 package com.example.LearningManagementSystem.Exception;
2
3 public class CourseMaterialNotFoundExcept extends RuntimeException{
4     public CourseMaterialNotFoundExcept(String msg) {
5         super(msg);
6     }
7 }
8
```

25 Current:

src\main\java\com\example\LearningManagementSystem\Exception\CourseNotFoundException.java

```
1 package com.example.LearningManagementSystem.Exception;
2
3 public class CourseNotFoundException extends RuntimeException{
4     public CourseNotFoundException(String message) {
5         super(message);
6     }
7     public CourseNotFoundException(String resourceName,String name){
8         super(resourceName + " not found with name: " + name);
9     }
10 }
11
```

26 Current:

src\main\java\com\example\LearningManagementSystem\Exception\DuplicateResourceException.java

```
1 package com.example.LearningManagementSystem.Exception;
2
3 public class DuplicateResourceException extends RuntimeException {
4     public DuplicateResourceException(String message) {
5         super(message);
6     }
7 }
8
```

27 Current:

src\main\java\com\example\LearningManagementSystem\Exception\ProfileAlreadySetException.java

```
1 package com.example.LearningManagementSystem.Exception;
2
3 public class ProfileAlreadySetException extends RuntimeException{
4     public ProfileAlreadySetException(String msg){
5         super(msg);
6     }
7 }
8
```

28 Current:

src\main\java\com\example\LearningManagementSystem\Exception\ProfileNotFoundException.java

```
1 package com.example.LearningManagementSystem.Exception;
2
3 public class ProfileNotFoundException extends RuntimeException{
4     public ProfileNotFoundException(String msg){
5         super(msg);
6     }
7 }
8
```


29 Current:

src\main\java\com\example\LearningManagementSystem\Exception\ResourceNotFoundException.java

```
1 package com.example.LearningManagementSystem.Exception;
2
3 public class ResourceNotFoundException extends RuntimeException {
4     public ResourceNotFoundException(String message) {
5         super(message);
6     }
7     public ResourceNotFoundException(String resourceName, Long id) {
8         super(resourceName + " not found with id: " + id);
9     }
10 }
11
```

30 Current:

src\main\java\com\example\LearningManagementSystem\Exception\StudentNotFoundException.java

```
1 package com.example.LearningManagementSystem.Exception;
2
3 public class StudentNotFoundException extends RuntimeException{
4     public StudentNotFoundException(String msg){
5         super(msg);
6     }
7 }
8
```

31 Current:

src\main\java\com\example\LearningManagementSystem\LearningManagementSystemApplication.java

```
1 package com.example.LearningManagementSystem;
2
3 import org.springframework.boot.SpringApplication;
4 import org.springframework.boot.autoconfigure.SpringBootApplication;
5 import org.springframework.cache.annotation.EnableCaching;
6
7 @SpringBootApplication
8 @EnableCaching
9 public class LearningManagementSystemApplication {
10
11     public static void main(String[] args) {
12         SpringApplication.run(LearningManagementSystemApplication.class, args);
13     }
14
15 }
16
```

32 Current:

src\main\java\com\example\LearningManagementSystem\Notification\EmailNotificationService.java

```
1 package com.example.LearningManagementSystem.Notification;
2
3 import org.springframework.context.annotation.Primary;
4 import org.springframework.stereotype.Service;
5
6 @Service
7 @Primary
8 public class EmailNotificationService implements NotificationService {
9
10     @Override
11     public void Notify(String message) {
12         System.out.println("Email sent!!:" + message);
13     }
14 }
```

```
1 package com.example.LearningManagementSystem.Notification;
2
3 public interface NotificationService {
4     void Notify(String message);
5 }
6
```

34 Current:

src\main\java\com\example\LearningManagementSystem\Notification\SmsNotificationService.java

```
1 package com.example.LearningManagementSystem.Notification;
2
3 import org.springframework.stereotype.Service;
4
5 @Service
6 public class SmsNotificationService implements NotificationService {
7     @Override
8     public void Notify(String message) {
9         System.out.println("SMS sent!!:" + message);
10    }
11 }
12
```

35 Current:

src\main\java\com\example\LearningManagementSystem\Projection\CourseMaterialCount.java

7

lines

```
1 package com.example.LearningManagementSystem.Projection;
2
3 public interface CourseMaterialCount {
4     String getCourseName();
5     Long getMaterialCount();
6 }
7
```

```
1 package com.example.LearningManagementSystem.Repository;
2
3 import com.example.LearningManagementSystem.Entity.Category;
4 import org.springframework.data.jpa.repository.JpaRepository;
5 import org.springframework.stereotype.Repository;
6
7 import java.util.Optional;
8
9 @Repository
10 public interface CategoryRepo extends JpaRepository<Category, Long> {
11     Optional<Category> findByName(String name);
12     boolean existsByName(String name);
13 }
14
```



```
1 package com.example.LearningManagementSystem.Repository;
2
3 import com.example.LearningManagementSystem.Entity.CourseMaterial;
4 import com.example.LearningManagementSystem.Projection.CourseMaterialCount;
5 import org.springframework.data.jpa.repository.JpaRepository;
6 import org.springframework.data.jpa.repository.Modifying;
7 import org.springframework.data.jpa.repository.Query;
8 import org.springframework.stereotype.Repository;
9
10 import java.util.List;
11
12 @Repository
13 public interface CourseMaterialRepo extends JpaRepository<CourseMaterial, Long> {
14
15     List<CourseMaterial> findByCourse_CourseId(long courseId);
16
17     // BUG FIX: original query was missing "FROM" keyword – broke at runtime
18     // Also @Modifying is not needed here (it's a SELECT), but added for any future UPDATE/DELETE queries
19     @Query("SELECT m.course.courseName AS courseName, COUNT(m.id) AS materialCount " +
20           "FROM CourseMaterial m GROUP BY m.course.courseName")
21     List<CourseMaterialCount> getMaterialCountPerCourse();
22
23     // Example of how @Modifying should be used – on any @Query that mutates data:
24     // @Modifying
25     // @Query("DELETE FROM CourseMaterial m WHERE m.course.courseId = :courseId")
26     // void deleteAllByCourseId(@Param("courseId") Long courseId);
27 }
```

```
1 package com.example.LearningManagementSystem.Repository;
2
3 import com.example.LearningManagementSystem.Entity.Course;
4 import org.springframework.data.jpa.repository.JpaRepository;
5 import org.springframework.stereotype.Repository;
6
7 import java.util.List;
8 import java.util.Optional;
9
10 @Repository
11 public interface CourseRepo extends JpaRepository<Course, Long> {
12     List<Course> findByInstructor_InstructorId(Long instructorId);
13     List<Course> findByCategory_CategoryId(Long categoryId);
14     List<Course> findByCourseNameContainingIgnoreCase(String keyword);
15     boolean existsByCourseName(String courseName);
16     Optional<Course> findByCourseNameIgnoreCase(String courseName);
17 }
18
```

```
1 package com.example.LearningManagementSystem.Repository;
2
3 import com.example.LearningManagementSystem.Entity.Enrollment;
4 import org.springframework.data.jpa.repository.JpaRepository;
5 import org.springframework.stereotype.Repository;
6
7 import java.util.List;
8 import java.util.Optional;
9
10 @Repository
11 public interface EnrollmentRepo extends JpaRepository<Enrollment, Long> {
12     List<Enrollment> findByStudent_StudentId(Long studentId);
13     List<Enrollment> findByCourse_CourseId(Long courseId);
14     Optional<Enrollment> findByStudent_StudentIdAndCourse_CourseId(Long studentId, Long courseId);
15     boolean existsByStudent_StudentIdAndCourse_CourseId(Long studentId, Long courseId);
16 }
17
```

```
1 package com.example.LearningManagementSystem.Repository;
2
3 import com.example.LearningManagementSystem.Entity.Instructor;
4 import org.springframework.data.jpa.repository.JpaRepository;
5 import org.springframework.stereotype.Repository;
6
7 import java.util.Optional;
8
9 @Repository
10 public interface InstructorRepo extends JpaRepository<Instructor, Long> {
11     Optional<Instructor> findByEmail(String email);
12     boolean existsByEmail(String email);
13 }
14
```

41 Current:

src\main\java\com\example\LearningManagementSystem\Repository\StudentProfileRepo.java

10
lines

```
1 package com.example.LearningManagementSystem.Repository;
2
3 import com.example.LearningManagementSystem.Entity.StudentProfile;
4 import org.springframework.data.jpa.repository.JpaRepository;
5 import org.springframework.stereotype.Repository;
6
7 @Repository
8 public interface StudentProfileRepo extends JpaRepository<StudentProfile, Long> {
9 }
10
```

```
1 package com.example.LearningManagementSystem.Repository;
2
3 import com.example.LearningManagementSystem.Entity.Student;
4 import org.springframework.data.jpa.repository.JpaRepository;
5
6 public interface StudentRepo extends JpaRepository<Student, Long> {
7 }
8
```

```
1 package com.example.LearningManagementSystem.Service;
2
3 import org.springframework.cache.Cache;
4 import org.springframework.cache.CacheManager;
5 import org.springframework.stereotype.Service;
6
7 import java.util.Objects;
8
9 @Service
10 public class CacheInspectionService {
11     private final CacheManager cacheManager;
12
13     public CacheInspectionService(CacheManager cacheManager) {
14         this.cacheManager = cacheManager;
15     }
16     public void printCache(String cachename){
17         Cache cache= cacheManager.getCache(cachename);
18         if(cache!=null){
19             System.out.println("Cache contents:");
20             System.out.println(Objects.requireNonNull(cache.getNativeCache().toString()));
21         }else{
22             System.out.println("No such cache:"+cachename);
23         }
24     }
25 }
26
```

```
1 package com.example.LearningManagementSystem.Service;
2
3 import com.example.LearningManagementSystem.Entity.Category;
4 import com.example.LearningManagementSystem.Exception.DuplicateResourceException;
5 import com.example.LearningManagementSystem.Exception.ResourceNotFoundException;
6 import com.example.LearningManagementSystem.Repository.CategoryRepo;
7 import org.springframework.stereotype.Service;
8 import org.springframework.transaction.annotation.Transactional;
9
10 import java.util.List;
11
12 @Service
13 @Transactional(readOnly = true) // Default all methods to read-only transactions
14 public class CategoryService {
15
16     private final CategoryRepo categoryRepository;
17
18     public CategoryService(CategoryRepo categoryRepository) {
19         this.categoryRepository = categoryRepository;
20     }
21
22     public List<Category> getAllCategories() {
23         return categoryRepository.findAll();
24     }
25
26     public Category getCategoryById(Long id) {
27         return categoryRepository.findById(id)
28             .orElseThrow(() -> new ResourceNotFoundException("Category", id));
29     }
30
31     @Transactional // Overrides class-level readOnly=true for write operations
32     public Category createCategory(Category category) {
33         if (categoryRepository.existsByName(category.getName())) {
34             throw new DuplicateResourceException("Category with name '" + category.getName() + "' already
exists.");
35         }
36         return categoryRepository.save(category);
37     }
38
39     @Transactional
40     public Category updateCategory(Long id, Category updated) {
41         Category existing = getCategoryById(id);
42         if (!existing.getName().equals(updated.getName()) &&
43             categoryRepository.existsByName(updated.getName())) {
44             throw new DuplicateResourceException("Category with name '" + updated.getName() + "' already
exists.");
45         }
46         existing.setName(updated.getName());
47         return categoryRepository.save(existing);
48     }
49
50     @Transactional
51     public void deleteCategory(Long id) {
52         getCategoryById(id);
53         categoryRepository.deleteById(id);
54     }
55 }
```



```

1 package com.example.LearningManagementSystem.Service;
2
3 import com.example.LearningManagementSystem.Entity.Course;
4 import com.example.LearningManagementSystem.Entity.CourseMaterial;
5 import com.example.LearningManagementSystem.Exception.CourseMaterialNotFoundException;
6 import com.example.LearningManagementSystem.Exception.CourseNotFoundException;
7 import com.example.LearningManagementSystem.Repository.CourseMaterialRepo;
8 import com.example.LearningManagementSystem.Repository.CourseRepo;
9 import org.springframework.http.HttpStatus;
10 import org.springframework.http.ResponseEntity;
11 import org.springframework.stereotype.Service;
12 import org.springframework.transaction.annotation.Transactional;
13
14 import java.util.List;
15
16 @Service
17 @Transactional(readOnly = true)
18 public class CourseMaterialService {
19
20     private final CourseMaterialRepo repo;
21     private final CourseRepo courseRepo;
22
23     public CourseMaterialService(CourseMaterialRepo repo, CourseRepo courseRepo) {
24         this.repo = repo;
25         this.courseRepo = courseRepo;
26     }
27
28     public List<CourseMaterial> getAllMaterials() {
29         return repo.findAll();
30     }
31
32     @Transactional
33     public CourseMaterial addMaterial(CourseMaterial material, String course_name) {
34         Course course = courseRepo.findByCourseNameIgnoreCase(course_name)
35             .orElseThrow(() -> new CourseNotFoundException("Course", course_name));
36         material.setCourse(course);
37         return repo.save(material);
38     }
39
40     public List<CourseMaterial> getMaterialById(long id) {
41         return repo.findByCourse_CourseId(id);
42     }
43
44     @Transactional
45     public void deleteMaterial(long id) {
46         // BUG FIX: was silently doing nothing if not found – now throws proper exception
47         CourseMaterial courseMaterial = repo.findById(id)
48             .orElseThrow(() -> new CourseMaterialNotFoundException("Course material with id:" + id + "
not found"));
49         repo.delete(courseMaterial);
50     }
51
52     public ResponseEntity<List<?>> getMaterialCount() {
53         return new ResponseEntity<>(repo.getMaterialCountPerCourse(), HttpStatus.OK);
54     }
55 }

```

```

1 package com.example.LearningManagementSystem.Service;
2
3 import com.example.LearningManagementSystem.Entity.Category;
4 import com.example.LearningManagementSystem.Entity.Course;
5 import com.example.LearningManagementSystem.Entity.Instructor;
6 import com.example.LearningManagementSystem.Exception.DuplicateResourceException;
7 import com.example.LearningManagementSystem.Exception.ResourceNotFoundException;
8 import com.example.LearningManagementSystem.Repository.CourseRepo;
9 import org.springframework.stereotype.Service;
10 import org.springframework.transaction.annotation.Transactional;
11
12 import java.time.LocalDateTime;
13 import java.util.List;
14
15 @Service
16 @Transactional(readOnly = true)
17 public class CourseService {
18
19     private final CourseRepo courseRepository;
20     private final InstructorService instructorService;
21     private final CategoryService categoryService;
22
23     public CourseService(CourseRepo courseRepository,
24                         InstructorService instructorService,
25                         CategoryService categoryService) {
26         this.courseRepository = courseRepository;
27         this.instructorService = instructorService;
28         this.categoryService = categoryService;
29     }
30
31     public List<Course> getAllCourses() {
32         return courseRepository.findAll();
33     }
34
35     public Course getCourseById(Long id) {
36         return courseRepository.findById(id)
37             .orElseThrow(() -> new ResourceNotFoundException("Course", id));
38     }
39
40     public List<Course> getCoursesByInstructor(Long instructorId) {
41         return courseRepository.findByInstructor_InstructorId(instructorId);
42     }
43
44     public List<Course> getCoursesByCategory(Long categoryId) {
45         return courseRepository.findByCategory_CategoryId(categoryId);
46     }
47
48     public List<Course> searchCourses(String keyword) {
49         return courseRepository.findByCourseNameContainingIgnoreCase(keyword);
50     }
51
52     @Transactional
53     public Course createCourse(Course course, Long instructorId, Long categoryId) {
54         if (courseRepository.existsByCourseName(course.getCourseName())) {
55             throw new DuplicateResourceException("Course with name '" + course.getCourseName() + "' already
56 exists.");
57         }
58         Instructor instructor = instructorService.getInstructorById(instructorId);
59         Category category = categoryService.getCategoryById(categoryId);
60         course.setInstructor(instructor);
61         course.setCategory(category);
62         course.setCreated_at(LocalDateTime.now());
63         return courseRepository.save(course);
64     }
65
66     @Transactional
67     public Course updateCourse(Long id, Course updated, Long instructorId, Long categoryId) {
68         Course existing = getCourseById(id);
69         if (!existing.getCourseName().equals(updated.getCourseName())
70             && courseRepository.existsByCourseName(updated.getCourseName())) {
71             throw new DuplicateResourceException("Course with name '" + updated.getCourseName() + "' already

```

```
exists.");
71     }
72     Instructor instructor = instructorService.getInstructorById(instructorId);
73     Category category = categoryService.getCategoryById(categoryId);
74     existing.setCourseName(updated.getCourseName());
75     existing.setDescription(updated.getDescription());
76     existing.setInstructor(instructor);
77     existing.setCategory(category);
78     return courseRepository.save(existing);
79 }
80
81 @Transactional
82 public void deleteCourse(Long id) {
83     getCourseById(id);
84     courseRepository.deleteById(id);
85 }
86 }
```

```

1 package com.example.LearningManagementSystem.Service;
2
3 import com.example.LearningManagementSystem.Entity.Course;
4 import com.example.LearningManagementSystem.Entity.Enrollment;
5 import com.example.LearningManagementSystem.Entity.Student;
6 import com.example.LearningManagementSystem.Exception.AlreadyEnrolledException;
7 import com.example.LearningManagementSystem.Exception.ResourceNotFoundException;
8 import com.example.LearningManagementSystem.Repository.EnrollmentRepo;
9 import com.example.LearningManagementSystem.Repository.StudentRepo;
10 import org.springframework.stereotype.Service;
11 import org.springframework.transaction.annotation.Transactional;
12
13 import java.time.LocalDateTime;
14 import java.util.List;
15
16 @Service
17 @Transactional(readOnly = true)
18 public class EnrollmentService {
19
20     private final EnrollmentRepo enrollmentRepository;
21     private final StudentRepo studentRepository;
22     private final CourseService courseService;
23
24     public EnrollmentService(EnrollmentRepo enrollmentRepository,
25                             StudentRepo studentRepository,
26                             CourseService courseService) {
27         this.enrollmentRepository = enrollmentRepository;
28         this.studentRepository = studentRepository;
29         this.courseService = courseService;
30     }
31
32     public List<Enrollment> getAllEnrollments() {
33         return enrollmentRepository.findAll();
34     }
35
36     public Enrollment getEnrollmentById(Long id) {
37         return enrollmentRepository.findById(id)
38             .orElseThrow(() -> new ResourceNotFoundException("Enrollment", id));
39     }
40
41     public List<Enrollment> getEnrollmentsByStudent(Long studentId) {
42         return enrollmentRepository.findByStudent_StudentId(studentId);
43     }
44
45     public List<Enrollment> getEnrollmentsByCourse(Long courseId) {
46         return enrollmentRepository.findByCourse_CourseId(courseId);
47     }
48
49     @Transactional
50     public Enrollment enroll(Long studentId, Long courseId) {
51         if (enrollmentRepository.existsByStudent_StudentIdAndCourse_CourseId(studentId, courseId)) {
52             throw new AlreadyEnrolledException("Student " + studentId + " is already enrolled in course " +
53                 courseId);
54         }
55         Student student = studentRepository.findById(studentId)
56             .orElseThrow(() -> new ResourceNotFoundException("Student", studentId));
57         Course course = courseService.getCourseById(courseId);
58
59         Enrollment enrollment = new Enrollment();
60         enrollment.setStudent(student);
61         enrollment.setCourse(course);
62         enrollment.setEnrolledDate(LocalDateTime.now());
63         return enrollmentRepository.save(enrollment);
64     }
65
66     @Transactional
67     public void unenroll(Long enrollmentId) {
68         getEnrollmentById(enrollmentId);
69         enrollmentRepository.deleteById(enrollmentId);
70     }
71 }

```

```
1 package com.example.LearningManagementSystem.Service;
2
3 import com.example.LearningManagementSystem.Entity.Instructor;
4 import com.example.LearningManagementSystem.Exception.DuplicateResourceException;
5 import com.example.LearningManagementSystem.Exception.ResourceNotFoundException;
6 import com.example.LearningManagementSystem.Repository.InstructorRepo;
7 import org.springframework.stereotype.Service;
8 import org.springframework.transaction.annotation.Transactional;
9
10 import java.util.List;
11
12 @Service
13 @Transactional(readOnly = true)
14 public class InstructorService {
15
16     private final InstructorRepo instructorRepository;
17
18     public InstructorService(InstructorRepo instructorRepository) {
19         this.instructorRepository = instructorRepository;
20     }
21
22     public List<Instructor> getAllInstructors() {
23         return instructorRepository.findAll();
24     }
25
26     public Instructor getInstructorById(Long id) {
27         return instructorRepository.findById(id)
28             .orElseThrow(() -> new ResourceNotFoundException("Instructor", id));
29     }
30
31     @Transactional
32     public Instructor createInstructor(Instructor instructor) {
33         if (instructorRepository.existsByEmail(instructor.getEmail())) {
34             throw new DuplicateResourceException("Instructor with email '" + instructor.getEmail() + "'
already exists.");
35         }
36         return instructorRepository.save(instructor);
37     }
38
39     @Transactional
40     public Instructor updateInstructor(Long id, Instructor updated) {
41         Instructor existing = getInstructorById(id);
42         if (!existing.getEmail().equals(updated.getEmail()) &&
instructorRepository.existsByEmail(updated.getEmail())) {
43             throw new DuplicateResourceException("Email '" + updated.getEmail() + "' is already in use.");
44         }
45         existing.setName(updated.getName());
46         existing.setEmail(updated.getEmail());
47         existing.setPassword(updated.getPassword());
48         existing.setSpecialization(updated.getSpecialization());
49         existing.setExperienceYears(updated.getExperienceYears());
50         return instructorRepository.save(existing);
51     }
52
53
54     @Transactional
55     public void deleteInstructor(Long id) {
56         getInstructorById(id);
57         instructorRepository.deleteById(id);
58     }
59 }
```

```
1 package com.example.LearningManagementSystem.Service;
2
3 import com.example.LearningManagementSystem.Entity.Student;
4 import com.example.LearningManagementSystem.Entity.StudentProfile;
5 import com.example.LearningManagementSystem.Exception.ProfileAlreadySetException;
6 import com.example.LearningManagementSystem.Exception.ProfileNotFoundException;
7 import com.example.LearningManagementSystem.Exception.StudentNotFoundException;
8 import com.example.LearningManagementSystem.Repository.StudentProfileRepo;
9 import com.example.LearningManagementSystem.Repository.StudentRepo;
10 import org.springframework.beans.factory.annotation.Autowired;
11 import org.springframework.stereotype.Service;
12 import org.springframework.transaction.annotation.Transactional;
13
14 import java.util.List;
15
16 @Service
17 @Transactional(readOnly = true)
18 public class StudentProfileService {
19
20     private final StudentProfileRepo studentProfileRepo;
21
22     public StudentProfileService(StudentProfileRepo studentProfileRepo) {
23         this.studentProfileRepo = studentProfileRepo;
24     }
25
26     @Autowired
27     private StudentRepo studentRepo;
28
29     public List<StudentProfile> getAllProfile() {
30         return studentProfileRepo.findAll();
31     }
32
33     @Transactional
34     public StudentProfile addProfile(StudentProfile studentProfile, long id) {
35         Student student = studentRepo.findById(id)
36             .orElseThrow(() -> new StudentNotFoundException("Student with id:" + id + " is not found"));
37
38         if (student.getProfile() != null) {
39             throw new ProfileAlreadySetException("Student with id:" + id + " has a profile");
40         }
41         student.setProfile(studentProfile);
42         studentProfile.setStudent(student);
43         Student updatedStudent = studentRepo.save(student);
44         return updatedStudent.getProfile();
45     }
46
47     @Transactional
48     public void deleteProfile(long id) {
49         StudentProfile profile = studentProfileRepo.findById(id)
50             .orElseThrow(() -> new ProfileNotFoundException("Profile with id:" + id + " not found"));
51         Student student = profile.getStudent();
52         if (student != null) {
53             student.setProfile(null);
54         }
55         studentProfileRepo.delete(profile);
56     }
57 }
```

```
1 package com.example.LearningManagementSystem.Service;
2
3
4
5 import com.example.LearningManagementSystem.DTO.StudentProfileDTO;
6 import com.example.LearningManagementSystem.Entity.Student;
7 import com.example.LearningManagementSystem.Entity.StudentProfile;
8 import com.example.LearningManagementSystem.Exception.StudentNotFoundException;
9 import com.example.LearningManagementSystem.Notification.NotificationService;
10 import com.example.LearningManagementSystem.Repository.StudentRepo;
11 import org.springframework.beans.factory.annotation.Qualifier;
12 import org.springframework.cache.Cache;
13 import org.springframework.cache.annotation.CacheEvict;
14 import org.springframework.cache.annotation.CachePut;
15 import org.springframework.cache.annotation.Cacheable;
16 import org.springframework.stereotype.Service;
17 import org.springframework.transaction.annotation.Transactional;
18 import org.springframework.transaction.support.TransactionSynchronization;
19 import org.springframework.transaction.support.TransactionSynchronizationManager;
20
21 import java.util.ArrayList;
22 import java.util.List;
23 import java.util.Optional;
24
25 @Service
26 @Transactional(readOnly = true)
27 public class StudentService {
28
29     private final StudentRepo studentRepo;
30     private final NotificationService notificationService;
31
32     public StudentService(StudentRepo studentRepo,
33                           @Qualifier("emailNotificationService") NotificationService notificationService) {
34         this.studentRepo = studentRepo;
35         this.notificationService = notificationService;
36     }
37
38     public List<StudentProfileDTO> getAllStudents() {
39         List<Student> students = studentRepo.findAll();
40         List<StudentProfileDTO> studentProfileDTOS = new ArrayList<>();
41         for (Student student : students) {
42             studentProfileDTOS.add(convertToStudentDTO(student));
43         }
44         return studentProfileDTOS;
45     }
46
47     @Cacheable(value = "Students",key = "#id")
48     public StudentProfileDTO getStudentById(long id){
49         Student student=studentRepo.findById(id)
50             .orElseThrow(()->new StudentNotFoundException("Student with id:" + id + " not found"));
51         return convertToStudentDTO(student);
52     }
53
54     @Transactional
55     @CacheEvict(value = "Students",key = "#id")
56     public StudentProfileDTO updateStudent(long id, Student updateStudent) {
57         var student = studentRepo.findById(id)
58             .orElseThrow(() -> new StudentNotFoundException("Student with id:" + id + " not found"));
59
60         if (updateStudent.getName() != null) student.setName(updateStudent.getName());
61         if (updateStudent.getEmail() != null) student.setEmail(updateStudent.getEmail());
62         if (updateStudent.getPassword() != null) student.setPassword(updateStudent.getPassword());
63         if (updateStudent.getPhoneNumber() != null) student.setPhoneNumber(updateStudent.getPhoneNumber());
64
65         if (updateStudent.getProfile() != null) {
66             StudentProfile existingProfile = student.getProfile();
67             if (existingProfile == null) {
68                 existingProfile = new StudentProfile();
69                 existingProfile.setStudent(student);
70                 student.setProfile(existingProfile);
71             }
72         }
73     }
74 }
```

```

72         StudentProfile incoming = updateStudent.getProfile();
73         if (incoming.getAddress() != null) existingProfile.setAddress(incoming.getAddress());
74         if (incoming.getDateOfBirth() != null)
existingProfile.setDateOfBirth(incoming.getDateOfBirth());
75         if (incoming.getEducation() != null) existingProfile.setEducation(incoming.getEducation());
76     }
77     Student savedStudent=studentRepo.save(student);
78     return convertToStudentDTO(savedStudent);
79 }
80
81 private StudentProfileDTO convertToStudentDTO(Student student) {
82     StudentProfileDTO studentProfileDTO = new StudentProfileDTO();
83     studentProfileDTO.setStudentId(student.getStudentId());
84     studentProfileDTO.setName(student.getName());
85     studentProfileDTO.setEmail(student.getEmail());
86     studentProfileDTO.setPhoneNumber(student.getPhoneNumber());
87     if (student.getProfile() != null) {
88         studentProfileDTO.setAddress(student.getProfile().getAddress());
89         studentProfileDTO.setDateOfBirth(student.getProfile().getDateOfBirth());
90         studentProfileDTO.setEducation(student.getProfile().getEducation());
91     }
92     return studentProfileDTO;
93 }
94
95 @Transactional
96 @Cacheable(value = "Students",key = "#id")
97 public StudentProfileDTO addStudent(Student student) {
98     StudentProfile studentProfile = student.getProfile();
99     if (studentProfile != null) {
100         notificationService.Notify("Profile created for id:"+student.getStudentId());
101         studentProfile.setStudent(student);
102         student.setProfile(studentProfile);
103     }
104     Student savedStudent = studentRepo.save(student);
105     return convertToStudentDTO(savedStudent);
106 }
107
108 @Transactional
109 @CacheEvict(value = "Students",key = "#id")
110 public void deleteStudent(long id) {
111     Student student = studentRepo.findById(id)
112         .orElseThrow(() -> new StudentNotFoundException("Student with id:" + id + " not found"));
113     studentRepo.delete(student);
114 }
115 }

```



```
1 spring:
2   application:
3     name: LearningManagementSystem
4
5   datasource:
6     url: jdbc:mysql://localhost:3306/lms
7     username: ${USER_NAME}
8     password: ${PASSWORD}
9
10  jpa:
11    hibernate:
12      ddl-auto: update
13    show-sql: true
```

52 Current:

src\test\java\com\example\LearningManagementSystem\LearningManagementSystemApplicationTests

```
1 package com.example.LearningManagementSystem;
2
3 import org.junit.jupiter.api.Test;
4 import org.springframework.boot.test.context.SpringBootTest;
5
6 @SpringBootTest
7 class LearningManagementSystemApplicationTests {
8
9     @Test
10     void contextLoads() {
11     }
12
13 }
14
```